CS509 Lab Work Guidelines

First-Year M.Tech CSE Students - 2026

1. Assignment Working Modes

Mode	Team Size	Requirement
Individual	1 student	The assignment is completed individually and stored in the student's Single repository.
Double / Buddy	2 students	The assignment is completed by a pair and stored in one repository shared by both students.
Group	3 or 4 students	The exact team size and repository arrangement will be decided and communicated later.

2. Programming Language

- Students may choose either C or C++. We would recommend C++.
- The selected language must be used consistently throughout the semester.
- Compilation and execution instructions in the README must match the selected language and toolchain.

3. GitHub Repository Requirements

Each student will maintain two repositories at this stage:

- One repository for all individual assignments.
- One shared repository for all Double / Buddy assignments completed by the pair.

3.1 Repository Naming Convention

Repository Type	Naming Pattern	Example
Individual	CS509_<EntryNumber>	CS509_CS1001
Double / Buddy	CS509_<EntryNumber1>_<EntryNumber2>	CS509_CS1001_AI1002

3.2 Recommended Repository Structure

The exact folder names may vary, but the repository must remain clear and consistent. A recommended structure is:

```
CS509_<EntryNumber>/
|-- README.md
|-- common_wrapper/
|   `-- wrapper.c or wrapper.cpp
|-- assignment_01/
|   |-- src/
|   |-- driver/
|   |-- tests/
|   |   |-- test_01.txt
|   |   |-- test_02.txt
|   |   `-- ...
|   `-- outputs/ (optional)
|-- assignment_02/
```

```
|   `-- ...
`-- ...
```

4. Assignment Implementation and Testing

4.1 Dedicated Driver for Every Assignment

- Every assignment must include a separate driver program or driver module.
- The driver must invoke the core algorithm and produce the required result and timing information.
- The driver should be kept separate from the core algorithm implementation wherever practical.
- The driver must support execution of one selected test file and/or all test files belonging to that assignment.

4.2 Test File Rules

- Every assignment will have multiple test files.
- Each test file must contain exactly one test case.
- Test files should use clear sequential names such as test_01.txt, test_02.txt, and test_03.txt.
- Every test case must be represented in the assignment's README result table.

4.3 Required Program Output

- The output for every test case must contain the computed result.
- The output must also contain the measured algorithm execution time, reported in milliseconds (ms) or seconds (s).
- The output format must be clear enough to distinguish the result from the timing value.

5. Runtime Measurement Rules

"Track time" means measuring only the execution time of the algorithm on a particular input. It does not mean the time spent by the student on implementation, debugging, or documentation.

- Timing must be performed for every input of every assignment.
- Only the algorithm execution time is to be reported.
- Input reading, file handling, input parsing, adjacency-list-to-CSR conversion, output formatting, and output printing must be outside the timed region.
- The timer should start immediately before the algorithm call and stop immediately after the algorithm finishes.
- For CSR-based graph algorithms, the CSR structure must already be prepared before the timer starts.
- The reported unit must be stated explicitly as ms or s.
- The compiler, relevant compilation flags, machine details, and timing method should be stated in the README so that the measurements can be interpreted correctly.

6. README Documentation Requirements

The README.md file is the complete documentation for the repository. The README must be exhaustive, clearly structured, and properly formatted.

6.1 Repository-Level Information

- Course and repository purpose.
- Student name and entry number; for pair work, the details of both students.
- Selected programming language, compiler, and required tool versions.

- Directory structure and the purpose of the main folders.
- Instructions for compiling and using the common wrapper.
- General conventions followed for test files, outputs, and runtime measurement.

6.2 Information Required for Every Assignment

- Assignment title, type (Single or Double / Buddy), and objective.
- Explanation of the algorithm or approach.
- Input format, assumptions, and constraints.
- Description of source files, driver files, helper functions, and test files.
- Exact compilation and execution instructions.
- Expected output, actual output, and algorithm execution time for every test case.
- Input sizes and other relevant properties, such as V and E for graphs or matrix dimensions for matrix problems.
- Time and space complexity, where applicable.
- References used for understanding or implementing the algorithm, where applicable.

6.3 Mandatory Result Table for Graph Problems

Every graph assignment, whether Single or Double / Buddy, must include a table covering every test case. The following columns are required:

Mode	**Test File**	**Input Type**	**Input Size**	**Expected Output**	**Actual Output**	**Algorithm Time**
Single / Double	test_01.txt	Adjacency list / graph type	V = __, E = __	________	________	________
Single / Double	test_02.txt	Adjacency list / graph type	V = __, E = __	________	________	________

For CSR-based implementations, the input type should mention that the adjacency list was converted to CSR before algorithm execution. The recorded timing must still contain only the algorithm execution time.

6.4 Mandatory Result Table for Matrix Problems

Every matrix assignment, whether Single or Double / Buddy, must include a table covering every test case. The following columns are required:

Mode	**Test File**	**Input Type**	**Input Size / Dimensions**	**Expected Output**	**Actual Output**	**Algorithm Time**
Single / Double	test_01.txt	Matrix input type	________	________	________	________
Single / Double	test_02.txt	Matrix input type	________	________	________	________

6.5 Suggested README Pattern

```
# CS509 Laboratory Repository

## Repository Overview
## Student / Pair Details
## Language and Environment
## Directory Structure
## Common Wrapper: Build and Usage

## Assignment 01 - <Title>
### Assignment Mode
### Objective
### Algorithm / Approach
### Input Format
### Helper Functions / CSR Conversion (if applicable)
### File Structure
### Compilation
### Execution
### Test Cases and Result Table
### Complexity
### References

## Assignment 02 - <Title>
...
```

7. Common Wrapper File

Every repository must include a common wrapper file or wrapper module that provides one menu or interface for accessing the submitted assignments.

- Display the list of available algorithms or assignments.
- Allow the user to select and compile one assignment.
- Allow the user to run one selected test file.
- Allow the user to run all test files for one assignment.
- Allow the user to compile and/or run all submitted algorithms, where feasible.
- Show clear error messages when a requested source file, test file, or executable is unavailable.

The common wrapper is the repository-level interface. It must invoke the dedicated driver belonging to the selected assignment and does not replace the separate driver required for each assignment.